

기획문서

문서 메타

항목	내용
서비스명	gohae (고해)
문서 버전	v1.0 — 프로덕션 출시 (12시간 스프린트)
최종 업데이트	@2026년 5월 16일
작성자 / 소유자	오준경 (jjuney)
프로젝트 형태	1인 사이드프로젝트 — 12시간 출시 스프린트
현재 단계	스코프·데이터모델 잠금 완료 → 부트스트랩 직전



v1.0 운영 원칙 — 단축된 일정이지, 단축된 품질이 아니다.

실제 사용자가 들어오는 프로덕트로 취급한다. 코드 품질·보안·UX·에러 처리는 "한 번 쓰고 버릴 수준"이 아니어야 한다. 시간 제약상 **빠는 것은 기능의 양이지 품질의 표준이 아니다.**

1. 프로젝트 개요

한 줄 정의 — gohae는 **장기 기억**과 **페르소나**를 가진 AI 상담 챗봇으로, 사용자의 맥락을 누적하여 "처음 보는 챗봇"이 아닌 "나를 아는 상대"로 작동하는 상담 서비스다.



비전 — 일상의 정서적 어려움을 털어놓을 적당한 상대가 없는 사람에게, 시간이 지나도 나를 기억하고 맥락을 이어주는 비대면 대화 창구를 제공한다.

2. 문제 정의 & 타겟

2.1 해결하려는 문제

- 일상적인 고민·정서적 어려움을 누군가에게 털어놓고 싶지만 **적당한 대상이 없는 상황** (가족/친구에게 부담 주기 싫음, 전문 상담은 진입 장벽 높음).

- 기존 범용 챗봇은 **매 세션마다 처음부터 다시 설명**해야 하고, 사용자의 인물관계·반복 이슈·트리거를 기억하지 못함.

2.2 타겟 사용자

- **1차 페르소나** — 정서적 부담을 가볍게 풀고 싶지만 주변에 적합한 대상이 없는 20-30대.
- **출시 시점의 초기 사용자** — 본인(오준경) 포함 좁은 초대 풀에서 시작. **그로스 단계가 아닌 검증 단계**임을 의식.

2.3 안 푸는 문제 (의도적 제외)

- 정신과 진단·치료 (의료기기 아님, 위기 신호 시 외부 자원으로 안내).
- 능동적 follow-up·푸시 알림.
- 다인 챗 / 그룹 상담.

3. 핵심 가치 / 차별점



핵심 차별점 두 가지에 집중한다. 그 외는 후순위.

1. **장기 기억 (Long-term Memory)** — 대화에서 사용자에게 대한 사실(인물관계, 반복 이슈, 감정 패턴, 트리거 등)을 LLM이 자동 추출하여 구조화 저장. 새 대화 시 관련 기억을 검색해 시스템 프롬프트에 주입.
2. **페르소나 온보딩** — 첫 진입 시 사용자가 직접 자기 정보를 입력하는 흐름. 초기 페르소나 시드로 활용.

왜 이 둘인가 — 기존 범용 챗봇(ChatGPT 등)이 **할 수는 있지만 잘 안 하는** 영역. 동일 LLM을 쓰더라도 "사용자 맥락 자산"이 누적되면 시간이 갈수록 차별이 커진다.

4. 제품 스코프 (v1.0)

4.1 v1.0에 들어가는 것

영역	구현 범위
인증	Google OAuth 단일
온보딩	3필드(닉네임 / 연령대 / 호칭 선호) + 자유 서술 1개
채팅	단일 스레드, Claude 스트리밍 응답, 메시지 영속화
장기 기억	"대화 끝내기" 버튼 → 동기 추출 → 다음 대화에 retrieval 주입
위기 안내	키워드 매칭 → 1393 카드 동기 노출
법적/신뢰 표면	이용약관·개인정보처리방침 정적 페이지, 푸터 링크
에러 모니터링	Sentry 연동 (FE/BE 양쪽)
배포	Vercel + Supabase 클라우드, HTTPS 기본

4.2 v1.0에 안 들어가는 것 (의식적 컷) ❌

영역	사유
이메일/비번 회원가입	메일 발송 인프라 부담, OAuth로 같음
다단계 온보딩 위저드, 이미지 업로드	검증 단계에서 마찰 추가 불필요
멀티 대화 스레드, 검색, 메시지 편집/삭제	v1.x로
비동기 잡 큐 (메모리 추출)	동기 호출로 동일 결과, 인프라 단순
모순 기억 자동 갱신	INSERT 중복 허용, retrieval top-K가 흡수. v1.x로
데이터 삭제 셀프 UI	요청 시 수동 cascade hard delete로 대응 (정책 문서에 명시)
감사 로그·통계 대시보드, ML 위기 분류	v1.x로
커스텀 도메인	시간 남으면

4.3 스코프 잠금 원칙



스프린트 중 "한 줄만 더 추가하면 되는데"는 가장 위험한 유혹. 시간이 남으면 새 기능이 아닌 폴리시(에러 메시지·로딩 상태·빈 화면·접근성)에 쓴다.

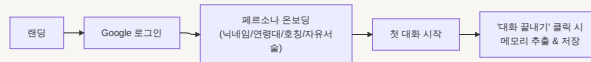
5. 핵심 사용자 플로우

5.1 신규 사용자 (Onboarding)

flowchart LR

```

A["랜딩"] --> B["Google 로그인"]
B --> C["페르소나 온보딩<br>(닉네임/연령대/호칭/자유서술)"]
C --> D["첫 대화 시작"]
D --> E["'대화 끝내기' 클릭 시<br>메모리 추출 & 저장"]
  
```



5.2 재방문 사용자 (Returning)

flowchart LR

```

A["로그인"] --> B["대화 진입"]
B --> C["최근 메시지 +<br>관련 장기기억 retrieval"]
C --> D["시스템 프롬프트 조립"]
D --> E["Claude API 스트리밍"]
E --> F["응답 표시 &<br>메시지 연속화"]
F --> G{"대화 끝내기<br>버튼?"}
G -->|"Yes"| H["메모리 추출 (동기)"]
G -->|"No"| B
H --> I["최근 메시지 +<br>관련 장기기억 retrieval"]
I --> J["시스템 프롬프트 조립"]
  
```

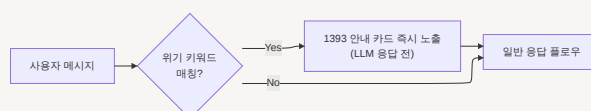


5.3 위기 신호 감지 플로우

flowchart LR

```

A["사용자 메시지"] --> B{"위기 키워드<br>매칭?"}
B -->|"Yes"| C["1393 안내 카드 즉시 노출<br>(LLM 응답 전)"]
B -->|"No"| D["일반 응답 플로우"]
C --> D
  
```





위기 안내는 **LLM 응답을 기다리지 않고** 동기적으로 먼저 노출한다. LLM이 위기 메시지를 잘못 응대할 위험을 차단하기 위함.

6. 핵심 시스템 설계 (v1.0)

6.1 장기기억 시스템

메모리 라이프사이클

flowchart TB

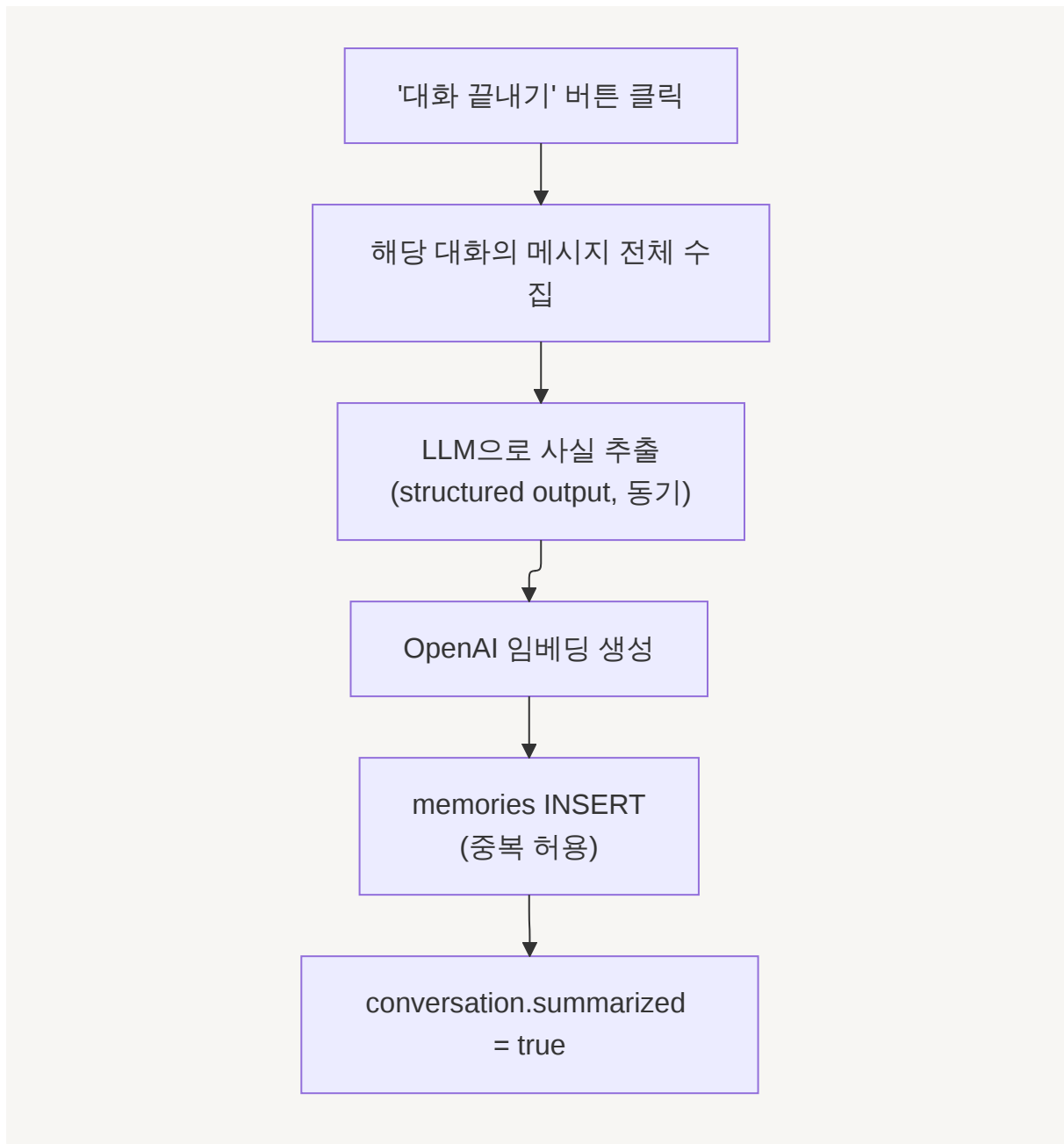
A["'대화 끝내기' 버튼 클릭"] --> B["해당 대화의 메시지 전체 수집"]

B --> C["LLM으로 사실 추출
(structured output, 동기)"]

C --> D["OpenAI 임베딩 생성"]

D --> E["memories INSERT
(중복 허용)"]

E --> F["conversation.summarized = true"]



기억 유형 (분류)

- `relationship` — 인물관계
- `recurring_issue` — 반복 이슈
- `emotional_pattern` — 감정 패턴
- `trigger` — 트리거
- `preference` — 사용자 선호
- `fact` — 기타 사실

Retrieval 전략 (v1.0)

1. 최근 사용자 메시지를 임베딩 → pgvector 코사인 유사도 top-K(K=5) 조회.
2. 토큰 예산 내에서 시스템 프롬프트에 주입.
3. 모순 처리·항상 포함 룰은 v1.0에 없음 — v1.x 작업.

6.2 페르소나 온보딩 (v1.0)

수집 항목 (확정)

- 필수:
 - 닉네임 (text)
 - 연령대 (10s | 20s | 30s | 40s+)
 - 호칭 선호 (formal 기본값 | casual)
- 선택: 자유 서술 1개 ("지금 가장 마음에 걸리는 일")
- 금지: 실명, 주소, 연락처 등 식별 정보

활용

- 시스템 프롬프트 고정 섹션으로 주입.
- 자유 서술은 첫 메모리 시드로 동일 파이프라인에 입력.

6.3 대화 인터페이스 (v1.0)

- 단일 활성 대화 — 사용자당 ended_at IS NULL 인 conversation은 항상 1개 이하.
- 메시지 영속화는 즉시 INSERT.
- 스트리밍 응답 (Vercel AI SDK useChat + Anthropic provider).
- "대화 끝내기" 버튼 = conversations.ended_at 세팅 + 메모리 추출 트리거.

6.4 신뢰성·관측 가능성 (v1.0 필수)



프로덕션 표준 = 사고를 빨리 알고 빨리 끄는 것. 사용자 1명이라도 들어오는 순간 부터 적용한다.

- 에러 모니터링: Sentry FE/BE 양쪽 연동. 메시지 원문은 캡처 금지(scrubbing 설정).

- 헬스체크: `/api/health` 엔드포인트 (Supabase ping + Anthropic ping).
- 외부 API 호출: 모두 timeout + 1회 재시도 + 실패 시 사용자에게 명시적 안내.
- 시크릿 관리: 모든 키는 Vercel/Supabase 환경변수로만 관리. 코드/로그에 노출 금지.
- HTTPS: Vercel + Supabase 기본 제공. 검증만 수행.

7. 데이터 모델 v1.0 — 4테이블



원칙 — 12시간 안에 만들고 운영해야 함. 컬럼 수·테이블 수 모두 미니멈. `users`는 Supabase Auth의 `auth.users`를 그대로 사용.

7.1 스키마 (SQL)

```
-- 1) personas: 1:1 with auth.users
create table personas (
  user_id      uuid primary key references auth.users(id) on
delete cascade,
  nickname     text not null,
  age_range    text not null,                -- '10s' | '20s' | '30s' | '40s+'
  tone         text not null default 'formal', -- 'formal' | 'casual'
  self_intro   text,                        -- 자유서술 1개
  created_at   timestampz default now(),
  updated_at   timestampz default now()
);

-- 2) conversations: 대화 단위 (메모리 추출의 grouping key)
create table conversations (
  id           uuid primary key default gen_random_uuid(),
  user_id      uuid not null references auth.users(id) on de
lete cascade,
  started_at   timestampz default now(),
  ended_at     timestampz,                  -- null = 진
행중
```

```

    summarized    boolean default false           -- 메모리 추출
완료 여부
);
create index on conversations(user_id, ended_at);

-- 3) messages
create table messages (
    id              bigserial primary key,
    conversation_id uuid not null references conversations(i
d) on delete cascade,
    role            text not null,                -- 'user' |
'assistant' | 'system'
    content         text not null,
    crisis_flag     boolean default false,
    created_at      timestampz default now()
);
create index on messages(conversation_id, created_at);

-- 4) memories: embedding 컬럼 포함
create extension if not exists vector;
create table memories (
    id              uuid primary key default gen_random_uuid(),
    user_id         uuid not null references auth.users(id) on de
lete cascade,
    type            text not null,                -- relationsh
ip|recurring_issue|emotional_pattern|trigger|preference|fac
t
    content         text not null,                -- 1-2문장 사
실
    embedding       vector(1536),                -- OpenAI tex
t-embedding-3-small
    status          text not null default 'active', -- 'active' |
'superseded'
    created_at      timestampz default now()
);
create index on memories using ivfflat (embedding vector_co
sine_ops) with (lists = 100);
create index on memories(user_id, status);

```

7.2 RLS 정책 (Supabase 필수)

모든 테이블에 RLS 활성화 + `user_id = auth.uid()` 단일 정책.

`conversations` · `messages` · `memories` 는 본인 것만, `personas` 는 본인 1행만. **RLS 미적용 상태**로는 절대 배포 금지.

7.3 v0.1 초안 대비 컷한 항목

- `memory_embeddings` 분리 테이블 → `memories.embedding` 컬럼으로 통합
- `memories.confidence` , `source_message_ids` , `superseded_by_id` 컬럼
- `last_summarized_at` (필요 시 `summarized` boolean으로 충분)

8. 안전장치 (양보 불가)



상담 도메인 = 윤리적 책임 도메인. v1.0이라도 이 섹션은 폴스펙으로 들어간다.

8.1 위기 신호 감지

- **1차 디텍션:** 키워드 매칭 (자살, 자해, "죽고 싶다", "사라지고 싶다" 등 사전 정의 리스트).
- **트리거 시 동작:**
 1. 사용자 메시지를 화면에 표시.
 2. **LLM 응답 전에** 1393 안내 카드를 시스템 메시지로 즉시 삽입.
 3. 이후 LLM에는 "위기 상황 인지하고 신중히 응대하라"는 추가 instruction을 system role로 주입.
 4. `messages.crisis_flag = true` 마킹.

8.2 1393 안내 카드 문구 (초안)

지금 많이 힘드신 것 같아요. 혼자 감당하기 어려운 마음이 들 땐, 전문 상담사와 익명으로 이야기할 수 있는 곳이 있어요. **자살예방상담전화 1393 (24시간, 무료)**

8.3 민감정보 처리

- 정신건강 관련 대화는 **민감정보**로 분류.

- 외부 전송: Claude API + OpenAI Embeddings API 외 **제3자에 원문 전송 금지**.
- 로깅: 에러 로그에 메시지 원문 포함 금지. 메시지 ID·메타데이터만. Sentry에는 scrubbing 적용.
- 데이터 삭제: v1.0엔 셀프 UI 없음. 정책 문서에 "요청 시 14일 내 완전 삭제" 명시 + 요청 채널 안내.

8.4 법적/신뢰 표면

- **이용약관·개인정보처리방침** 정적 페이지 (`/terms` , `/privacy`).
 - 수집 항목, 보관 기간, 제3자 제공(Anthropic, OpenAI) 명시.
 - "본 서비스는 의료기기가 아니며 진단·치료를 제공하지 않음" 고지.
- **푸터·온보딩**에 두 페이지 링크 노출 + 회원가입 시 동의 체크박스.
- 초안은 LLM으로 생성 후 본인 검토. 추후 법률 검토는 사용자 확보 후 별도 트랙.

9. 기술 스택 (v1.0 확정)

9.1 확정 스택

영역	선택	비고
풀스택 프레임워크	Next.js 15 (App Router) • TypeScript	Route Handlers + Server Actions 로 BE 흡수
DB + 인증 + pgvector	Supabase	region: ap-northeast-2 (서울)
ORM	Drizzle ORM	Prisma보다 가벼움, 마이그레이션 즉시
UI	Tailwind CSS + shadcn/ui	디자인 시간 최소화, 접근성 기본
채팅 스트리밍	Vercel AI SDK (<code>ai</code> , <code>@ai-sdk/anthropic</code>)	<code>useChat</code> 혹
LLM (대화·추출)	Claude Sonnet 4.6	품질 우선
임베딩	OpenAI <code>text-embedding-3-small</code> (1536d)	Anthropic 임베딩 API 없음
호스팅	Vercel	<code>vercel deploy</code>
에러 모니터링	Sentry (필수)	FE/BE 양쪽, 원문 scrubbing
디자인 보조	<code>v0.app</code> / shadcn 블록	디자이너 없이 모던

9.2 v0.1 초안 대비 변경 (Django → Next.js)



큰 결정: Django + DRF를 **v1.0에서 제외**. 12시간 + Next.js 조합에서 BE 분리
는 환경변수·CORS·세션 분리 비용이 과대. Next.js Route Handlers로 BE 흡
수. Django는 v1.x 이후 도메인 복잡도 증가 시 재검토.

9.3 금지 (1인 운영 부담 과대)

- AWS ECS/CloudFront 라인
- Kubernetes / 마이크로서비스 / 자체 호스팅 메시지 큐
- 학습 곡선이 큰 신규 기술 (gRPC, GraphQL)

10. 12시간 타임박스



T+0 = 스프린트 시작. 사전 준비(§14) 완료 후 타이머 시작.

T+0:00 ~ 0:30 기획 **잠금** (현 문서 최종 확인) + 위기 키워드 v1 리
스트 작성

T+0:30 ~ 1:30 디자인 - shadcn 블록 픽 + v0.app 채팅 **UI**/온보
딩 폼/약관 페이지

T+1:30 ~ 2:30 부트스트랩 - create-next-app, Supabase **마이그
레이션(+RLS)**, env, Sentry, Vercel link, 첫 deploy

T+2:30 ~ 3:30 인증 - Supabase Auth Google OAuth + 미들웨어
+ 동의 체크박스

T+3:30 ~ 5:30 채팅 - Vercel **AI SDK** + Claude + messages 저
장 + 타임아웃/재시도

T+5:30 ~ 7:00 페르소나 온보딩 + 시스템 프롬프트 조립

T+7:00 ~ 9:30 장기기억 v1 - 추출 프롬프트 + 임베딩 + retrieva
l

T+9:30 ~ 10:30 위기 키워드 + **1393 카드** (LLM 응답 전 동기)

T+10:30 ~ 11:00 약관/개인정보처리방침 페이지 + 푸터 + 헬스체크 엔
드포인트

T+11:00 ~ 11:45 본인 **E2E** 사용 **테스트** (골든패스 + 엡지 2-3개) +
핵심 버그 fix

T+11:45 ~ 12:00 최종 deploy + 운영 점검 (Sentry 이벤트 흐름, 환경변수 누락 체크)

10.1 각 구간 완료 정의

- **T+2:30** — Vercel에 빈 페이지가 떠 있고, Supabase 4테이블 + RLS 적용 완료. Sentry에 deploy 이벤트 보임.
- **T+3:30** — Google 로그인 → 약관 동의 → 빈 대시보드 도달.
- **T+5:30** — 로그인 → 채팅 → Claude 응답 스트리밍 → messages DB에 저장. API 실패 시 사용자에게 안내 표시.
- **T+7:00** — 신규 가입 시 온보딩 폼 → personas INSERT → 채팅 응답에 톤·호칭 반영.
- **T+9:30** — "대화 끝내기" → memories INSERT → 새 대화 시작 시 이전 기억이 시스템 프롬프트에 보임.
- **T+10:30** — 위기 키워드 포함 메시지 입력 시 1393 카드 즉시 노출.
- **T+11:00** — `/terms` · `/privacy` 접근 가능, 푸터 링크 동작, `/api/health` 200.
- **T+12:00** — 외부 URL에서 본인이 end-to-end 동작 확인 + Sentry 대시보드에 의도된 에러 1건 캡처 검증.

10.2 시간 부족 시 컷 순서 (역우선순위)

미완 시 이 순서로 잘라낸다 — 위기 안내·약관·기본 채팅은 절대 컷 금지.

1. 커스텀 도메인 → 11번째로 컷
2. 헬스체크 엔드포인트 → 10번째
3. UI 폴리시 (애니메이션, 빈 상태 그림) → 9번째
4. 메모리 추출 프롬프트 추가 튜닝 → 8번째
5. 페르소나 자유 서술 필드 → 7번째
6. (그 아래는 모두 양보 불가 영역)

10.3 합정 3가지 (미리 피하기)

1. **Google OAuth redirect URI** — Supabase 대시보드의 정확한 callback URL을 Google Cloud Console에 등록. 0:30~2:30 셋업 단계에서 끝낼 것.

2. **Supabase pgvector extension** — 대시보드 → Database → Extensions에서 활성화 확인. 마이그레이션 실패 방지.
3. **메모리 추출은 동기**로 — 비동기 잡 큐 도입 금지. "대화 끝내기" 클릭 시 그 자리에서 LLM 호출 → INSERT.

11. 리스크 & (잠긴) 의사결정

11.1 운영 리스크

리스크	영향	완화책
OAuth 설정 지연	1~2시간 손실	사전 준비(\$14) 완료 후 시작
메모리 추출 품질 낮음	차별점이 죽음	추출 프롬프트 1회 반복 개선만 (최대 1시간), 그 이상은 v1.x
Claude/OpenAI API 비용	운영 중 quota 소진	사전 충전, 토큰 cap, 사용자 수 제한
시간 초과	출시 미완성	\$10.2 컷 순서 적용
위기 키워드 누락/오탐	윤리 사고	v1.0은 룰 기반으로 보수적, 의심 시 false positive 쪽으로
Sentry에 메시지 원문 유출	민감정보 사고	scrubbing 설정 + deploy 직후 실제 캡처 점검

11.2 잠긴 의사결정

- ✓ FE: **Next.js (App Router) + TypeScript**
- ✓ 인증: **Google OAuth 단일 (Supabase Auth)**
- ✓ 인프라: **Vercel + Supabase**
- ✓ BE 분리 여부: **v1.0은 Next.js 풀스택 (Django 제외)**
- ✓ 메모리 추출 트리거: **명사 버튼 only**
- ✓ Claude 모델: **Sonnet 4.6**
- ✓ 호칭 기본값: **존댓말 (formal)**
- ✓ 서비스명: **gohae 유저**
- ✓ 페르소나 필수 항목: **닉네임 / 연령대 / 호칭 + 자유서술 1개**
- ✓ 에러 모니터링: **Sentry 필수**

✓ 약관·개인정보처리방침: v1.0 포함

□ 위기 키워드 v1 리스트 (T+0:00~0:30 구간에 외부 자료 참고하여 1차 작성)

12. 결정 로그



재논의 방지용 결정 기록. 새로운 결정적 근거 없이는 재오픈하지 않는다.

일자	결정	근거
@2026년 5월 16일	서비스명: gohae 유지	종교적 함의는 차별점
@2026년 5월 16일	핵심 차별점: 장기 기억 + 페르소나 온보딩	1인 개발 제약상 차별점 분산 금지
@2026년 5월 16일	인프라: Vercel + Supabase . AWS 라인 제외	1인 운영 부담
@2026년 5월 16일	능동성·CBT 프레임워크 등 추가 차별점 후보 제외	스코프 보호
@2026년 5월 16일	위기 안내는 LLM 응답 전 동기적 노출	LLM 단독 응대의 위험 차단
@2026년 5월 16일	v1.0에서 Django 제외, Next.js 풀스택	12시간 + BE 분리 = 환경변수·CORS·세션 분리 비용 과대
@2026년 5월 16일	DB·인증·pgvector를 Supabase로 통합	셋업 5분, 자체 호스팅 부담 0
@2026년 5월 16일	ORM은 Drizzle	Prisma보다 가벼움
@2026년 5월 16일	UI는 Tailwind + shadcn/ui	디자인 비용 최소화, 접근성 기본
@2026년 5월 16일	채팅 스트리밍은 Vercel AI SDK	<code>useChat</code> 혹 즉시
@2026년 5월 16일	임베딩은 OpenAI text-embedding-3-small	Anthropic 임베딩 API 없음
@2026년 5월 16일	Claude 모델: Sonnet 4.6	품질 우선
@2026년 5월 16일	메모리 추출 트리거: "대화 끝내기" 버튼 동기 호출	12시간엔 비동기 큐 사치, 동기로도 동일 결과
@2026년 5월 16일	데이터 모델: 4테이블 (personas, conversations, messages, memories)	<code>auth.users</code> 는 Supabase 제공, embedding은 memories 컬럼으로 흡수
@2026년 5월 16일	페르소나 필수 항목: 닉네임/연령대/호칭 + 자유서술 1개	온보딩 마찰 최소화

일자	결정	근거
@2026년 5월 16일	호칭 기본값: 존댓말(formal)	안전한 디폴트
@2026년 5월 16일	데이터 삭제 셀프 UI는 v1.x 로, v1.0은 정책문서로 대응	시간 제약, 정책 명시로 신뢰 확보
@2026년 5월 16일	Sentry 필수, 약관·개인정보처리방침 v1.0 포함	프로덕션 출시 표준

13. 부록 A — v1.x 이후 로드맵

v1.0 출시 후 다시 끌어올 사항. **스프린트 중에는 절대 끌어오지 않는다.**

- Django + DRF로 BE 재분리 검토 (도메인 복잡도 증가 시)
- 메모리 모순 자동 갱신 (`superseded_by_id` 부활)
- 페르소나 동적 보강 (관찰 기반)
- 비동기 잡 큐 (메모리 추출을 background로)
- Retrieval 평가 데이터셋 + 정량 평가
- 능동적 follow-up (조심스럽게)
- 데이터 삭제 셀프 UI
- 멀티 대화 스레드, 검색
- 약관·개인정보처리방침 법률 검토

14. 부록 B — 출시 전 사전 준비 체크리스트



T+0 이전에 끝내야 할 30분 작업. 미완 시 T+0:30~2:30 구간이 막힌다.

- ☐ **Supabase** 계정 + 빈 프로젝트 1개 (region: ap-northeast-2 서울 권장)
- ☐ **Google Cloud Console** OAuth 2.0 클라이언트 ID 발급
 - Authorized redirect URI: `https://<your-project>.supabase.co/auth/v1/callback`
(Supabase 대시보드 Auth → Providers → Google에서 정확한 URL 확인)
- ☐ **Anthropic Console** API 키 발급 + \$20 충전
- ☐ **OpenAI Platform** API 키 발급 + \$5 충전

- ☐ **Vercel** 계정 + GitHub 연동
 - ☐ **GitHub** private repo 생성 (gohae)
 - ☐ **Sentry** 프로젝트 생성 (Next.js)
-



다음 액션 — §14 사전 준비를 마치면 T+0 시작. 2단계 = 레포 부트스트랩 + Supabase 마이그레이션(+RLS) + Sentry 연동 + 첫 Vercel deploy.